Batch: B2	Roll No.: 16010124107
Experiment No. 5	

Title: Queries based Views and Triggers

Objective: To be able to use SQL view and triggers.

Expected Outcome of Experiment:

CO3: Utilize SQL for Relational Database Operations

Books/ Journals/ Websites referred:

1. Dr. P.S. Deshpande, SQL and PL/SQL for Oracle 10g.Black book, Dreamtech Press
2. www.db-book.com
3. Korth, Slberchatz, Sudarshan : "Database Systems Concept", 5th Edition , McGraw Hill
4. Elmasri and Navathe,"Fundamentals of database Systems", 4th Edition,PEARSON Education.

Resources used: Postgresql

Theory

View

Views are pseudo-tables. That is, they are not real tables; nevertheless appear as ordinary tables to SELECT. A view can represent a subset of a real table, selecting certain columns or certain rows from an ordinary table. A view can even represent joined tables. Because views are assigned separate permissions, you can use them to restrict table access so that the users see only specific rows or columns of a table.

A view can contain all rows of a table or selected rows from one or more tables. A view can be created from one or many tables, which depends on the written PostgreSQL query to create a view.

Types of Views

Simple View: Created from a single table without using any aggregate functions (like SUM(), AVG()) or grouping.

Complex View: Created from multiple tables, often using joins, grouping, or aggregate functions.

Views, which are kind of virtual tables, allow users to do the following –

- Structure data in a way that users or classes of users find natural or intuitive.
- Restrict access to the data such that a user can only see limited data instead of a complete table.
- Summarize data from various tables, which can be used to generate reports.

Views with joins or aggregations are often "read-only" and cannot be updated directly.

Syntax

CREATE [TEMP | TEMPORARY] VIEW view_name AS

SELECT column1, column2.....

FROM table_name

WHERE [condition];

Ex:

CREATE VIEW HR_Staff AS

SELECT EmpID, Name, Department

FROM Employees

WHERE Department = 'HR';

Display view:

select * from Company-View

Insert record into view:

Insert into Company-View values (123,'alpha', 10)

select * from Company

Dropping Views

Syntax: DROP VIEW view_name;

Updating a View Definition

A. If we want to update the existing data within the view, use the UPDATE statement.

UPDATE view_name

SET column1 = value1, column2 = value2...., columnN = valueN

WHERE [condition];

B. If you want to update the view definition without affecting the data, use the CREATE OR REPLACE VIEW statement.

For example, let's add the Age column to the MarksView:

CREATE OR REPLACE VIEW view_name AS

SELECT column1, column2, …

FROM table_name

WHERE condition;

Note: Not all views can be updated using the UPDATE statement.

Create view from multiple table

Syntax:

CREATE VIEW View_name AS

SELECT table1.Col1, table1.col2, table2.co1…

FROM table1

JOIN table2

ON table1.common_column = table2.common_column;

Example

CREATE VIEW EmployeeDeptInfo AS

SELECT E.Name, D.DeptName

FROM Employees E

JOIN Departments D ON E.DeptID = D.DeptID;

Triggers

A trigger is a stored procedure in a database that automatically invokes whenever a special event in the database occurs. By using SQL triggers, developers can automate tasks, ensure data consistency, and keep accurate records of database activities.

For example, a trigger can be invoked when a row is inserted into a specified table or when specific table columns are updated.

Key Features of SQL Triggers:

Automatic Execution: Triggers fire automatically when the defined event occurs (e.g., INSERT, UPDATE, DELETE).

Event-Driven: Triggers are tied to specific events that take place within the database.

Table Association: A trigger is linked to a specific table or view, and operates whenever changes are made to the table's data.

The **basic syntax of creating a trigger** is as follows –

```
CREATE  TRIGGER trigger_name [BEFORE|AFTER|INSTEAD OF] event_name
ON table_name
[
 -- Trigger logic goes here....
];
```

- **trigger_name:** The name of the trigger to be created.
- **BEFORE | AFTER:** Specifies whether the trigger is fired before or after the triggering event (INSERT, UPDATE, DELETE).
- **{INSERT | UPDATE | DELETE}**: Specifies the operation that will activate the trigger.
- **table_name:** The name of the table the trigger is associated with.

- **FOR EACH ROW:** Indicates that the trigger is row-level, meaning it executes once for each affected row. (You can optionally specify FOR EACH ROW after table name.
-)
- **trigger_body:** The SQL statements to be executed when the trigger is fired.

The following is the syntax of creating a trigger on an UPDATE operation on one or more specified columns of a table as follows –

CREATE TRIGGER trigger_name [BEFORE|AFTER] UPDATE OF column_name

ON table_name

[

-- Trigger logic goes here....

];

To use the PostgreSQL CREATE TRIGGER statement to create a trigger.

To create a new trigger in PostgreSQL, you follow these steps:

- First, create a trigger function using CREATE FUNCTION statement.
- Second, bind the trigger function to a table by using CREATE TRIGGER statement.

Example 1 :

Suppose that when the name of an employee changes, you want to log it in a separate table called **employee_audits.**

employee

```
emp_id | fname |  lname   | age |  salary
--------+-------+---------+-----+----------
      1 | John  | Doe     |  30 | 50000.00
      2 | Jane  | Smith   |  25 | 60000.00
      3 | Alice | Johnson |  35 | 70000.00
(3 rows)
```

Employee_audit

```
emp_id | fname |  ChangeOn  |

--------+------+---------
```

CREATE OR REPLACE FUNCTION log_last_name_changes()

```
 RETURNS TRIGGER AS $$
BEGIN
 INSERT INTO employee_audits(employee_id,last_name,changed_on)
        VALUES(OLD.id,OLD.last_name,now());
        RETURN NEW;
END;
$$ language plpgsql;

CREATE TRIGGER last_name_changes
  BEFORE UPDATE
  ON employees
  FOR EACH ROW
  EXECUTE PROCEDURE log_last_name_changes();
select * from employee;

UPDATE employee
SET lname = 'Kulkarni'
WHERE ID = 1

select * from semployee
select * from employee_audit
```

Example 2:
In the agriculture database ,do not allow harvest entry before sowing date.

Tables-

- **Planting** (planting_id, sowing_date, …)

- **Harvest** (harvest_id, harvest_date, yield_quantity, planting_id)

Trigger Objective- To ensure Harvest date must be after sowing date.

Trigger Type - BEFORE INSERT trigger on Harvest table

Step 1: Create Trigger Function

```
CREATE OR REPLACE FUNCTION check_harvest_date()
RETURNS TRIGGER AS $$
DECLARE
   sow_date DATE;
BEGIN
   -- Fetch sowing date for the planting
   SELECT sowing_date
   INTO sow_date
   FROM Planting
   WHERE planting_id = NEW.planting_id;

   -- Validation rule
   IF NEW.harvest_date <= sow_date THEN
      RAISE EXCEPTION 'Harvest date must be after sowing date';
   END IF;

   RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Step 2: Create Trigger on Harvest Table

```
CREATE TRIGGER trg_check_harvest_date
BEFORE INSERT ON Harvest
FOR EACH ROW
```

EXECUTE FUNCTION check_harvest_date();

Testing the Trigger

Valid insert:

INSERT INTO Harvest (harvest_id, harvest_date, yield_quantity, planting_id)

VALUES (1, '2025-03-20', 1200, 101);

Invalid Insert:

INSERT INTO Harvest (harvest_id, harvest_date, yield_quantity, planting_id)

VALUES (2, '2025-01-05', 800, 101);

DROP TRIGGER

DROP TRIGGER is used to permanently remove a trigger from a table.

Once dropped:

- Trigger will not fire
- Trigger logic won't execute
- Table operations continue normally

Syntax

DROP TRIGGER trigger_name ON table_name;

OR

DROP TRIGGER IF EXISTS trigger_name ON table_name; (Prevents error if trigger does not exist).

Implementation Screenshots (Problem Statement, Query and Screenshots of Results):

```
-- 1 customers + accounts
SELECT C.name, A.acc_no, A.balance
FROM customer C
LEFT JOIN Accounts A ON C.acc_no = A.acc_no;
```

Data Output Messages Notifications

	name character varying (100)	acc_no character varying (50)	balance double precision
1	Rahul	ACC1	5000
2	Amit	ACC2	6000
3	Neha	ACC3	7000
4	Pooja	ACC4	8000
5	Raj	ACC5	9000
6	Karan	ACC6	10000
7	Simran	ACC7	11000
8	Arjun	ACC8	12000
9	Meera	ACC9	13000
10	Vikram	ACC10	14000

```
-- 2 customers + loans
SELECT C.name, L.loan_id, L.amt
FROM customer C
LEFT JOIN LOANS L ON C.loan_id = L.loan_id;
```

	name character varying (100)	loan_id character varying (30)	amt double precision
1	Rahul	L1	10000
2	Amit	L2	20000
3	Neha	L3	15000
4	Pooja	L4	18000
5	Raj	L5	22000
6	Karan	L6	25000
7	Simran	L7	30000
8	Arjun	L8	12000
9	Meera	L9	17000
10	Vikram	L10	19000

```
-- 3 accounts + customer
SELECT C.name, A.acc_no, A.balance
FROM customer C
RIGHT JOIN Accounts A ON C.acc_no = A.acc_no;
```

	name character varying (100)	acc_no character varying (50)	balance double precision
1	Rahul	ACC1	5000
2	Amit	ACC2	6000
3	Neha	ACC3	7000
4	Pooja	ACC4	8000
5	Raj	ACC5	9000
6	Karan	ACC6	10000
7	Simran	ACC7	11000
8	Arjun	ACC8	12000
9	Meera	ACC9	13000
10	Vikram	ACC10	14000

```
-- 4 transactions + account
SELECT A.acc_no, T.transaction_id, T.amt
FROM Accounts A
RIGHT JOIN transaction T ON A.acc_no = T.acc_no;
```

	acc_no character varying (50)	transaction_id integer	amt double precision
1	ACC1	1	500
2	ACC2	2	600
3	ACC3	3	700
4	ACC4	4	800
5	ACC5	5	900
6	ACC6	6	1000
7	ACC7	7	1100
8	ACC8	8	1200
9	ACC9	9	1300
10	ACC10	10	1400

```
-- 5 full join
SELECT C.name, A.acc_no
FROM customer C
FULL OUTER JOIN Accounts A ON C.acc_no = A.acc_no;
```

	name character varying (100)	acc_no character varying (50)
1	Rahul	ACC1
2	Amit	ACC2
3	Neha	ACC3
4	Pooja	ACC4
5	Raj	ACC5
6	Karan	ACC6
7	Simran	ACC7
8	Arjun	ACC8
9	Meera	ACC9
10	Vikram	ACC10

-- 6 total balance
SELECT SUM(balance) FROM Accounts;

Data Output Messages Notificatio

	sum double precision
1	95000

-- 7 avg balance
SELECT AVG(balance) FROM Accounts;

	avg double precision
1	9500

-- 8 max min balance
SELECT MAX(balance), MIN(balance) FROM Accounts;

	max double precision	min double precision
1	14000	5000

-- 9 count customers
SELECT COUNT(*) FROM customer;

	count bigint
1	10

-- 10 accounts by type
SELECT account_type, COUNT(*)
FROM Accounts
GROUP BY account_type;

	account_type character varying (20)	count bigint
1	Savings	6
2	Current	4

-- 11 total transaction per account
SELECT acc_no, SUM(amt)
FROM transaction
GROUP BY acc_no;

	acc_no character varying (50)	sum double precision
1	ACC6	1000
2	ACC10	1400
3	ACC8	1200
4	ACC3	700
5	ACC2	600
6	ACC1	500
7	ACC7	1100
8	ACC9	1300
9	ACC4	800
10	ACC5	900

```
-- 12 avg salary by dept
SELECT department, AVG(salary)
FROM Employee
GROUP BY department;
```

	department character varying (100)	avg numeric
1	IT	30800.000000000000
2	HR	26600.000000000000

```
-- 13 account types >2
SELECT account_type, COUNT(*)
FROM Accounts
GROUP BY account_type
HAVING COUNT(*) > 2;
```

	account_type character varying (20)	count bigint
1	Savings	6
2	Current	4

```
-- 14 accounts >10000 txn
SELECT acc_no, SUM(amt)
FROM transaction
GROUP BY acc_no
HAVING SUM(amt) > 10000;
```

acc_no character varying (50)	sum double precision

-- 15 highest balance
SELECT C.name, A.balance
FROM customer C
JOIN Accounts A ON C.acc_no = A.acc_no
WHERE A.balance = (SELECT MAX(balance) FROM Accounts);

	name character varying (100)	balance double precision
1	Vikram	14000

-- 16 second highest
SELECT MAX(balance)
FROM Accounts
WHERE balance < (SELECT MAX(balance) FROM Accounts);

Data Output Messages Notifications

	name character varying (100)	balance double precision
1	Meera	13000

-- 17 salary > avg
SELECT *
FROM Employee
WHERE salary > (SELECT AVG(salary) FROM Employee);

Showing rows: 1 to 5 Page No: 1 of 1

	emp_id [PK] integer	salary integer	hire_date date	name character varying (100)	department character varying (100)	phone_no character varying (15)	designation character varying (100)	ifsc character varying (20)
1	1	30000	2023-01-01	Aman	IT	9000000001	Manager	IFSC001
2	5	35000	2023-01-05	Karan	IT	9000000005	Lead	IFSC005
3	7	29000	2023-01-07	Arjun	IT	9000000007	Dev	IFSC007
4	8	31000	2023-01-08	Meera	HR	9000000008	Manager	IFSC008
5	9	32000	2023-01-09	Vikram	IT	9000000009	Lead	IFSC009

-- 18 customers who made payment
SELECT DISTINCT C.name
FROM customer C
JOIN payment P ON C.customer_id = P.payer_customer_id;

	name character varying (100)
1	Rahul
2	Neha
3	Raj
4	Simran
5	Arjun
6	Pooja
7	Amit
8	Meera
9	Vikram
10	Karan

```
-- 19 accounts no transaction
SELECT *
FROM Accounts A
WHERE NOT EXISTS (
  SELECT 1 FROM transaction T
  WHERE T.acc_no = A.acc_no
);
```

Data Output Messages Notifications

acc_no [PK] character varying (50)	balance double precision	created_at date	account_type character varying (20)	status character varying (20)	ifsc character varying (20)

```
-- 20 customers with loan + account
```

```
SELECT DISTINCT C.name
FROM customer C
WHERE C.acc_no IS NOT NULL AND C.loan_id IS NOT NULL;
```

	name character varying (100)
1	Rahul
2	Neha
3	Raj
4	Simran
5	Arjun
6	Pooja
7	Amit
8	Meera
9	Vikram
10	Karan

Conclusion:

Queries executed successfully for joins, aggregation, and subqueries, enabling efficient data retrieval, analysis, and validation across multiple related tables in the database.

Post Lab Questions:

1. What is a view in SQL, and how does it differ from a table?

A view is a virtual table based on query results. It stores no data itself. A table stores actual data physically. Views simplify queries, improve security, and present specific data, while tables hold complete data permanently in the database.

2. Write a query to create a complex view that display Fname, Dept_id, and Salary from the employee table and dept name from Dept table.

```
CREATE VIEW emp_dept_view AS

SELECT E.Name AS Fname,

    E.Department AS Dept_id,

    E.Salary,

    A.Address AS Dept_name

FROM Employee E

JOIN admin A ON E.ifsc = A.ifsc;
```

3. Write a query to drop the trigger trg_city_update from the employee table.

```
DROP TRIGGER trg_city_update ON Employee;
```